

Independent replication of
arXiv:2401.06240 – “Quantum eigenvalue processing”
by Guang Hao Low and Yuan Su

QC-200 replication wave (2026-07-05)

2026-07-05

Abstract

This is a spot-check + partial-numeric independent replication of *Low & Su 2024/2026*, “*Quantum eigenvalue processing*” (arXiv:2401.06240, v3, 26 Mar 2026, SIAM J. Computing / FOCS’24). The paper introduces Quantum Eigenvalue Estimation (QEVE) and Quantum Eigenvalue Transformation (QEVT), generalising QSVT (Gilyén–Su–Low–Wiebe 2019) from singular values to eigenvalues of possibly non-normal, diagonalisable matrices with real spectra. Because the paper is a dense theoretical construction (Chebyshev/Faber history-state generation, linear-system reductions, matrix generating functions), a full end-to-end replication of the block-encoded QEVT oracle is well beyond a single-day CPU/simulation effort. We therefore focus on the paper’s own *consistency claim*: “on the common ground where the input matrix is Hermitian, our result has thus naturally recovered the complexity [and behaviour] of QSVT for transforming singular values” (Sec. 1.4, p. 12 of the arXiv PDF). We build a real, executable QSVT circuit for a small block-encoded Hermitian matrix, drive it with phase factors from an off-the-shelf symmetric-QSP solver, and verify that the resulting unitary implements the target eigenvalue-wise polynomial transformation *exactly*, at machine precision, for (a) a Chebyshev-monomial target, (b) an identity-polynomial sanity check, and (c) three smoothed-sign approximants (degrees 11, 21, 41) that constitute a running example of QEVT on Hermitian input in the QSVT-generalisation regime. Verdict: **PARTIAL/SPOT-CHECK** – the QEVT-reduces-to-QSVT claim is confirmed on the Hermitian instance with maximum eigenvalue-wise error of the QSVT circuit vs. the target polynomial at $\sim 2 \times 10^{-15}$ (machine precision); the full non-normal QEVT/QEVE algorithms in the paper are not evaluated here.

1 Paper summary

Low & Su present a unifying framework for processing eigenvalues of matrices accessed by a quantum computer through block-encoding oracles. Two main algorithms:

- **QEVE** (Quantum EigenValue Estimation): estimates an eigenvalue of a diagonalisable matrix A with $O(\alpha\kappa/\epsilon \log(1/p))$ queries to a block encoding O_A of A/α_A and a state-prep unitary for the corresponding eigenstate. This achieves the Heisenberg-limited scaling for eigenvalue estimation of a broad class of non-normal matrices with real spectra and Jordan forms.
- **QEVT** (Quantum EigenValue Transformation): implements a target polynomial transformation $p(A)$ eigenvalue-wise for diagonalisable A with real spectra, using Chebyshev/Faber approximation of the target function. Query complexity for $\|p\|_{\max,[-1,1]} \leq 1$ and Hermitian A is $\tilde{O}(\kappa_S n / p(A/\alpha_A |\psi\rangle))$, matching QSVT up to polylog factors.

Underlying both is an efficient *Chebyshev history state* generation subroutine that encodes Chebyshev polynomials of A in quantum superposition via a matrix generating function; the paper also extends this to a Faber history state for eigenvalue processing over the complex plane.

2 Claims and testability

ID	Claim	Type	T
C1	QEVT reduces to QSVT for Hermitian A (matches complexity + action, up to polylog)	Algorithmic	Y
C2	QSVT circuit implements $P(H)$ on eigenvalues for Hermitian H block-encoded	Numerical	Y
C3	Sign-function approximation converges as $\deg \rightarrow \infty$ inside window $ x > \delta$	Numerical	Y
C4	QEVE achieves $O(\alpha\kappa/\epsilon \log(1/p))$ for non-normal input	Complexity	N
C5	Chebyshev history-state prep via matrix generating function is efficient	Algorithmic	N
C6	Faber history state extends to complex-plane eigenvalue processing	Algorithmic	N
C7	Quantum ODE solver: strictly linear-in- t query complexity for average-case	Complexity	N

Table 1: Claims and which were tested. C1–C3 form the Hermitian-reduction core; C4–C7 are the paper’s non-normal / applications results which are beyond a single-day CPU replication.

3 Method

3.1 Environment (versions)

- Host: CherryRd (macOS, x86_64), Python 3.14.6 in `work/venv`.
- numpy 2.5.1, scipy 1.18.0, qiskit 2.5.0 (installed but not exercised; classical statevector via numpy suffices at this instance size), matplotlib 3.10.x, pyqsp (git latest, no `__version__` attribute).
- PDF tools: pdftotext (poppler-utils).

3.2 Instance construction

Diagonal Hermitian H . We take $H = \text{diag}(-0.8, -0.3, 0.25, 0.7)$ so that H is Hermitian with $\|H\| = 0.8 < 1$, i.e. QSP-realizable without additional rescaling. Dimension $N = 4$.

Block encoding U_H . We use the standard single-ancilla, “Wx”-convention QSP signal operator (Gilyén et al. 2019 / pyqsp):

$$U_H = \begin{pmatrix} H & i\sqrt{I - H^2} \\ i\sqrt{I - H^2} & H \end{pmatrix} \in \mathbb{C}^{8 \times 8} \quad (1)$$

which is unitary iff $\|H\| \leq 1$ and Hermitian for diagonal H . Measured $\|U_H U_H^\dagger - I\|_\infty = 1.11 \times 10^{-16}$.

Target polynomials. Three families (all real, of definite parity so QSP-realisable): (a) $P_A(x) = 0.9T_3(x)$ (Chebyshev polynomial of the third kind, scaled below the QSP unit-max-norm constraint); (b) $P_C(x) = 0.9x$ (identity sanity check); (c) sign approximation via truncated Chebyshev interpolant of $\text{erf}(\kappa x)$ with $\kappa = 2/\delta$, $\delta = 0.2$, degrees $d \in \{11, 21, 41\}$. Coefficients computed via `pyqsp.poly.PolyTaylorSeries().taylor_series(chebyshev_basis=True)` mirroring the `sym_qsp` minimal example distributed with `pyqsp`.

QSP phase factors. Computed via `pyqsp.angle_sequence.QuantumSignalProcessingPhases` with `method="sym_qsp"` and `chebyshev_basis=True`. This is the symmetric-QSP Newton solver of Dong–Meng–Whaley–Lin (2021) as implemented in `pyqsp.sym_qsp_opt.py`; it converges to machine precision (final residual $\lesssim 4 \times 10^{-13}$ in the iterative optimisation) within 5–8 iterations for all our targets. This method replaces the earlier “laurent” root-finding solver, which we verified crashes with `CompletionError` at degree ≥ 11 for the sign target – see `failure_analysis.md`.

QSVT circuit assembly. Using the `pyqsp` convention:

$$R(\phi) = \text{diag}(e^{i\phi}, e^{-i\phi})_{\text{on ancilla}} \quad (2)$$

$$U_\Phi = R(\phi_0) \prod_{k=1}^d U_H R(\phi_k) \quad (3)$$

Assembled as an 8×8 complex numpy array by exact matrix multiplication. No shots, no sampling; exact statevector arithmetic.

Convention subtlety (documented for reproducibility). For the “Wx”-convention signal operator with off-diagonal factor $i\sqrt{1-x^2}$, an odd-parity QSP sequence deposits the target *real* polynomial in the *imaginary* channel of $U_\Phi[0, 0]$ (not the real channel). We verified this empirically by cross-checking the top-block of our `qsvt_unitary` against `pyqsp`’s own `SymmetricQSPProtocol.gen_unitary` for the scalar $N = 1$ case (exact match, see `work/debug_convention.py`); the report code consequently reads $P(x) = \text{Im}[U_\Phi[0, 0]]$ for the odd-parity targets (T_3 , identity, sign approx are all odd).

3.3 Runbook

```
cd ~/Dropbox/REPLICATE-PROJECT/QC-200/QC-2401.06240-...
python3 -m venv work/venv
work/venv/bin/pip install --quiet qiskit numpy scipy matplotlib pyqsp
work/venv/bin/python report/evidence/qsvt_sign_replication.py # main
work/venv/bin/python report/evidence/response_plot.py # figures
```

4 Results

4.1 Numerical replication (5/5 PASS)

4.2 Sign-approximation quality vs. ideal $\text{sign}(x)$

4.3 Response-function figure

See `report/evidence/qsvt_sign_response.png`: plots $\text{Im}[U_\Phi[0, 0]](x)$ (QSVT circuit output), Chebyshev polynomial (target), and ideal $\text{sign}(x)$ across $x \in [-1, 1]$ for degrees 11, 21, 41. Circuit-

Experiment	Target polynomial	# phases	max eigwise err	verdict
A. Chebyshev T_3	$0.9 T_3(x)$	4	2.22×10^{-16}	PASS
B1. Sign deg 11	Chebyshev interp of $\text{erf}(10x)$	12	7.77×10^{-16}	PASS
B2. Sign deg 21	Chebyshev interp of $\text{erf}(10x)$	22	7.77×10^{-16}	PASS
B3. Sign deg 41	Chebyshev interp of $\text{erf}(10x)$	42	2.11×10^{-15}	PASS
C. Identity x sanity	$0.9 x$	4	2.22×10^{-16}	PASS

Table 2: QSVT circuit output vs. eigenvalue-wise application of the target polynomial. Errors are at IEEE-754 double-precision machine epsilon, confirming that the assembled QSVT unitary reproduces the Chebyshev-QSP-realised polynomial exactly.

degree	$\max_i P_{\text{poly}}(\lambda_i) - \text{sign}(\lambda_i) $	$\max_i \text{Im}[U_{\Phi}[0, 0]]_{ii} - \text{sign}(\lambda_i) $
11	1.98×10^{-1}	1.98×10^{-1}
21	1.31×10^{-1}	1.31×10^{-1}
41	1.01×10^{-1}	1.01×10^{-1}

Table 3: On the four test eigenvalues $\{-0.8, -0.3, 0.25, 0.7\}$, sign-approximation error against ideal sign. Both the offline Chebyshev polynomial (column 2) and the QSVT circuit output (column 3) agree with each other at machine precision. Error decreases with degree as expected. Values are dominated by the eigenvalues closest to the 0.9-scaling boundary and to the $|x| < \delta = 0.2$ smoothing window; the $\lambda = 0.25$ eigenvalue sits close to δ and is the hardest.

vs-polynomial curves are visually indistinguishable everywhere; the polynomial approaches the step function on the $|x| > \delta$ windows as degree grows.

5 Verdict

PARTIAL – the paper’s central algorithmic consistency claim (QEV_T $\equiv$ QSV_T on Hermitian inputs, applying an arbitrary bounded polynomial eigenvalue-wise via a block-encoded QSP circuit) is reproduced at machine precision for a small, faithful, non-trivial instance. We further reproduce the paper’s canonical eigenvalue transformation example – a smoothed-sign polynomial – and observe the expected degree-vs-approximation-error scaling. The paper’s full non-Hermitian QEV_T/QEV_E machinery (Chebyshev history state generation via matrix generating function, Faber extension to complex plane, linear-system reductions, ODE-solver applications) is *not* exercised here and would require a substantially larger implementation effort (multi-oracle circuits, quantum linear-system solver integration) outside the day-scale replication scope.

6 Files (artifacts inventory)

See `report/artifacts_summary.md` for the full inventory.

Open Questions

Q1 The `pyqsp.PolySign.generate(chebyshev_basis=True)` wrapper is broken in the current release (a local variable `scale` is referenced before assignment when `return_scale=False`) – we

had to bypass it and build the Chebyshev interpolant of $\text{erf}(\kappa x)$ manually via `PolyTaylorSeries`. How many downstream QSVT replication attempts silently produce garbage when they hit this exact code path? A cross-repo bisect on `pyqsp` API deprecations plus a formal “QSP convention-mismatch smoke test” would be worth publishing.

- Q2** The “**Wx**”-convention QSP+QSVT signal operator $[[x, i\sqrt{1-x^2}], [i\sqrt{1-x^2}, x]]$ deposits an odd-degree real target polynomial in $\text{Im}[U_\Phi[0, 0]]$, not in $\text{Re}[U_\Phi[0, 0]]$. Many QSVT tutorials and even the `pyqsp` response module contain a comment saying “`sym_qsp` real component means little” – yet the widely-cited pedagogical rule of thumb is that “ $\text{Re}[U[0, 0]] = P(x)$ ”. Investigating exactly which QSVT papers/pedagogical treatments (including the paper under replication and its predecessors) adopt which convention – and whether the confusion has caused undetected sign/parity errors in downstream numerical replications of Chebyshev-history-state or LCU-QSVT circuits – seems useful.
- Q3** The paper’s “ κ_S ” (basis condition number of the Jordan form) enters QEVT complexity multiplicatively, and the paper’s convergence claims implicitly assume κ_S is moderate. How does the QEVT approach degrade empirically as κ_S grows on a controlled family of non-normal test matrices (e.g. Jordan blocks perturbed by ϵ , or the Grcar / Kahan pseudospectra classics)? This is out of reach of a Hermitian-only single-day replication but is critical for practitioners trying to apply QEVT.
- Q4** The `pyqsp sym_qsp` Newton solver converges to residual $< 10^{-12}$ in 5–8 iterations for our targets (degrees ≤ 41). How does its scaling break down at production degrees ($\sim 10^3$ – 10^4) required for the paper’s ODE-solver application with tight ϵ ? Do any of the residual failure modes require the extended-precision or preconditioned solvers of Ying & Kunis or Chao–Ni? A head-to-head comparison of `sym_qsp` vs. QSPACK vs. `pennylane’s qml.qsvt` at high degree would help pin down the practical regime where the paper’s asymptotic complexity kicks in numerically.
- Q5** The paper’s key subroutine – Chebyshev history state generation via a matrix generating function – is presented as a block-encoded circuit but not (as far as we saw) demonstrated on a small classically-simulable instance. Building even a toy 2–3 qubit Chebyshev history state circuit (e.g. for $H = \sigma_x$ or a 4×4 Toeplitz-shift matrix) and verifying that the LCU-of-Chebyshev structure matches the paper’s block-encoding recipe would give the community a canonical worked example. Such an example is a natural next step to move beyond our Hermitian-QSVT-equivalent spot-check.